

Introduction to Serialization and Deserialization

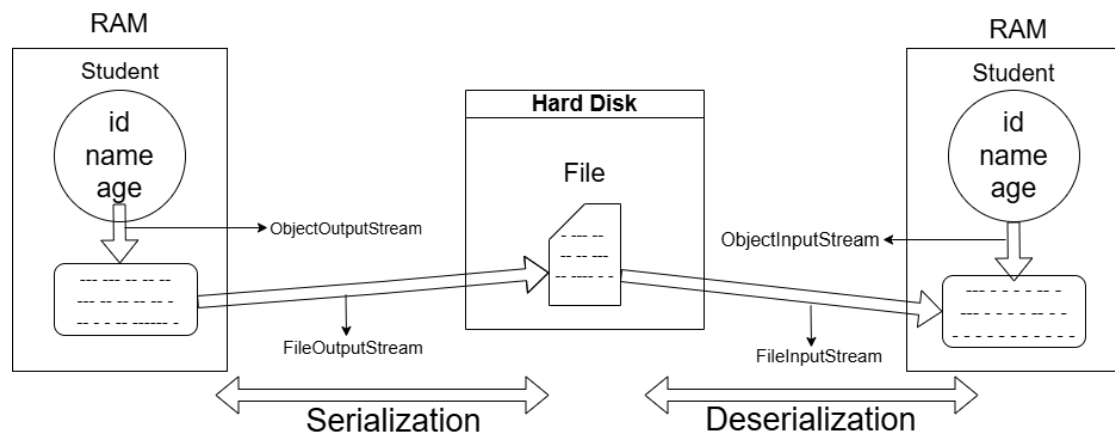
👉 What is Serialization and Deserialization?

Serialization

Serialization is the process of converting a Java object into a stream of bytes to store it in a file, database, or transmit over a network. When serialized, the object's state is saved in a non-readable format, providing security since anyone accessing the file cannot easily understand its contents.

Deserialization

Deserialization is the reverse process - converting the byte stream back into a Java object that can be used in our program.



👉 Serialization Process

- We have a Student object in RAM with attributes like id, name, and age
- The object exists in heap memory (RAM)
- To achieve persistence (saving the object before memory is deallocated), we convert it to a non-understandable byte format
- This conversion is handled by **ObjectOutputStream**
- The bytes are then written to a file on the hard disk using **FileOutputStream**

👉 Deserialization Process

- Later, we want to retrieve our Student object
- We read the byte data from the file using `FileInputStream`
- We convert those bytes back into a Java object using `ObjectInputStream`
- The Student object is reconstructed in RAM with all its properties (id, name, age)

👉 Key Points

- ✚ We cannot use regular file handling classes like `FileWriter` or `FileReader` for serialization
- ✚ We must use streams: `FileInputStream`, `FileOutputStream`, `ObjectInputStream`, and `ObjectOutputStream`
- ✚ Any class whose objects need to be serialized **must implement the Serializable interface**
- ✚ Serializable is a **marker interface** (it has no methods to implement)
- ✚ It signals to the JVM that this class's objects can be converted to byte streams
- ✚ At runtime, JVM adds serialization functionality to these classes
- ✚ Serialization provides security by storing objects in a non-understandable format
- ✚ Deserialization reconstructs the original object with all its properties